



**Tecnológico
de Monterrey**

Desarrollo de aplicaciones avanzadas de ciencias computacionales (Gpo 503)

Tarea 1

DOCENTE

Elda Quiroga

Ana Karina Aramoni Ruíz

A07192068

1 de Octubre del 2026

Stack (LIFO)

Implementé el Stack con un vector (`std::vector`), que almacena los elementos de forma contigua en memoria, haciendo el acceso más rápido. Además, el vector maneja su propia memoria, por lo que no se necesitan `new` ni `delete`. Como `push`, `pop` y `top` operan al final del vector, son $O(1)$. Se importó `<vector>` para lograr esto.

Test cases: El happy path verifica que `top` regrese el último elemento, que `pop` siga el orden LIFO y que `top` consulte sin eliminar. Los edge cases usan un solo elemento, 100,000 elementos, reutilización tras vaciar, `INT_MIN/INT_MAX` y duplicados. Los casos vacíos y nulos verifican que `pop` y `top` lancen `out_of_range` en un Stack vacío, ya vaciado o limpiado con `clear`, y que se guarden "" y `nullptr`.

Queue (FIFO)

El Queue elimina elementos del inicio, lo que en un vector normal obligaría a desplazar todos los demás. Para evitarlo, se mueve el marcador del frente (`head_`) en vez de los elementos, y al llegar al final del vector la fila se cicla al inicio usando el operador módulo:

```
if (size_ == buf_.size()) grow();
buf_[(head_ + size_) % buf_.size()] = value;
++size_;
```

Si el vector se llena, `grow()` duplica su capacidad y copia los elementos en el orden de la fila.

Test cases: El happy path verifica el orden FIFO y que `front` no elimine. Los edge cases validan el buffer circular, su capacidad exacta sin crecer, el noveno elemento que activa `grow()`, el ciclado al inicio del vector y `grow()` con la fila ciclada; además de 100,000 elementos y reutilización. Los casos vacíos y nulos verifican que `dequeue` y `front` lancen `out_of_range` y que se guarden "" y `nullptr`.

Hash

La tabla hash es un vector de buckets, donde cada bucket es un vector de pares clave-valor (`std::pair<K, V>`). `std::hash` convierte la clave en un número, y el módulo lo reduce al índice del bucket, por lo que las operaciones son $O(1)$ en promedio

```
std::size_t index_for(const K& key) const {
    return std::hash<K>{}(key) % buckets_.size();
}
```

Las colisiones se resuelven por encadenamiento, donde las claves que caen en el mismo bucket se guardan juntas y se buscan una por una. Cuando hay más elementos que buckets, `grow()` los duplica y reacomoda cada par. Para eliminar, se copia el último par del bucket en el hueco, evitando desplazamientos. `find` regresa un apuntador al valor, o `nullptr` si la clave no existe.

Test cases: `Search` corresponde a `find` y `delete` a `erase`, ya que `delete` es una palabra reservada de C++. El happy path verifica guardar, buscar, actualizar sin duplicar, `contains` y eliminar solo la clave indicada. Los edge cases validan que las claves se encuentren tras `grow()` y con 10,000 elementos, colisiones forzadas, eliminación en medio de un bucket compartido, reinserción, distinción entre mayúsculas y minúsculas, y claves como 0, negativos, `INT_MAX` e `INT_MIN`. Los casos vacíos y nulos verifican que buscar o eliminar claves inexistentes regrese `nullptr` o `false` sin errores, y que `""` funcione como clave y como valor. Al final, al guardar `nullptr` como valor, `find` regresa un apuntador válido hacia él, lo que distingue “la clave existe con valor vacío” de “la clave no existe”.

Uso de IA

Utilicé la inteligencia artificial de Claude para generar casos de prueba de las estructuras de datos implementadas. El prompt utilizado es el siguiente:

“Generate test cases for the following data structure classes and their respective methods in C++:

- 1. Stack (push, pop, top)*
- 2. Queue (enqueue, dequeue, front)*
- 3. Hash (put, search, contains, delete)*

For each, include the happy path, boundary values, and empty / null data.”

Debido a que C++ es un lenguaje que aún no domino, realicé la siguiente prompt para aprender acerca de optimización y mejores prácticas:

“Analyze and identify potential areas where the code could be optimized / be more industry standard, and state why. Walk me through the thought process.”